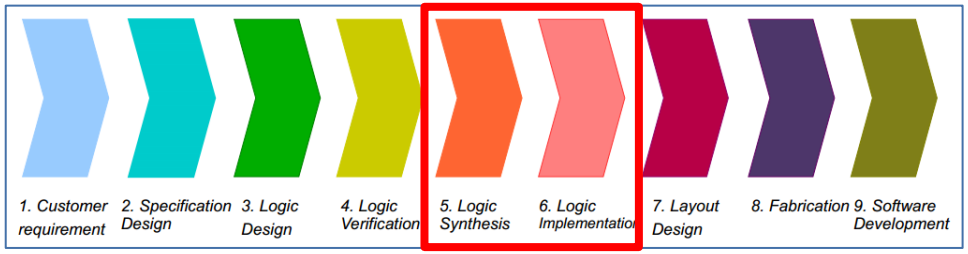
BKU / RVC / CADENCE COLLABORATION

LAB 3B – LOGIC EQUIVALENCE CHECK

RENESAS

Overview

- In LSI design flow, after Synthesis, we need to perform Logic Equivalence checking (LEC) between RTL and its Netlist to ensure that the functionality of them are the same:



Phases that Logic Equivalence checking is performed

- This Lab will familiarize you with one of Cadence tool – **Conformal-LEC**, which perform LEC.
- This Lab will cover the following:
  - ❑ **How to use Conformal-LEC**
  - ❑ **How to debug Non-equivalent points by using GUI (Graphic User Interface)**

Advantages of Conformal-LEC

- Conformal-LEC has many product to adapt with your purpose:
  - Conformal L (ASIC): Verify synthesized and place & route netlist transformation.
  - Conformal XL (Ultra): Conformal L + Verify complex datapath.
  - Conformal GXL (Custom) : Conformal XL + Verify custom logic and custom memories.
  - Conformal Low Power: Conformal XL + Verify low-power logic and power domains.
- It's self-contained and it not tied to any particular synthesis environment. Thus, it gives you a higher degree of confidence than equivalence checkers integrated with a particular logic synthesis tool.
- It has excellent debugging capabilities. It automatically diagnoses design mismatches and accurately pinpoints the source of the differences.
- It supports Hierarchical Comparison, which speed up the compare process. Moreover, Cadence already introduced Smart LEC with special engine, which can execute hierarchical comparison in parallel to achieve the best run time.

Perform LEC with Conformal (1/4)

- **Step 1:** Change directory to "*lec_env*" folder. In this Lab 4, we will work at this place:

```
%> cd /home/vlsi_group??/vlsi/${Student_ID}/work/lec_env
```

- **Step 2:** Link the RTL, Netlist and Library file from *"synthesis_env*" into this place:

```
%> ln -sf ../synthesis_env/Genus_BoundFlasher/RTL/bound_flasher.v
%> ln -sf ../synthesis_env/Genus_BoundFlasher/LAB1/outputs??/bound_flasher_m.v
%> ln -sf ../synthesis_env/Genus_BoundFlasher/LIB/slow.lib
```

- **Step 3:** Confirm there has no broken link (link files successfully):

```
%> ll
```

```
[rvc@localhost lec_env]$ ll
total 0
lrwxrwxrwx. 1 rvc rvc 81 Jul 30 11:45 bound_flasher_m.v -> ../synthesis_env/Genus_BoundFlasher/LAB1/outputs_Jul30-11:43:46/bound_flasher_m.v
lrwxrwxrwx. 1 rvc rvc 55 Jul 29 16:12 bound_flasher.v -> ../synthesis_env/Genus_BoundFlasher/RTL/bound_flasher.v
lrwxrwxrwx. 1 rvc rvc 48 Jul 29 16:13 slow.lib -> ../synthesis_env/Genus_BoundFlasher/LIB/slow.lib
```

Perform LEC with Conformal (2/4)

- **Step 4:** Prepare the setup script for Conformal as below:

```
%> vi ./lec.tcl
```

lec.tcl

```
set_log_file lec.log -replace

read_library slow.lib -lib -revised

read_design bound_flasher.v -verilog -golden

read_design bound_flasher_m.v -verilog -revised

set_mapping_method -name only
set_system_mode lec
map_key_points

add_compared_points -all
compare
```

Set Log file name

Read libraries

Read RTL

Read Netlist

Mapping process

Compare process

Perform LEC with Conformal (3/4)

- **Step 5:** Prepare the execution script as below:

```
%> vi ./go_lec
```

go_lec

```
#!/bin/bash -f

cd /home/share_file/cadence
source add_path
source add_license
cd -

lec -64 -dofile ./lec.tcl &
```

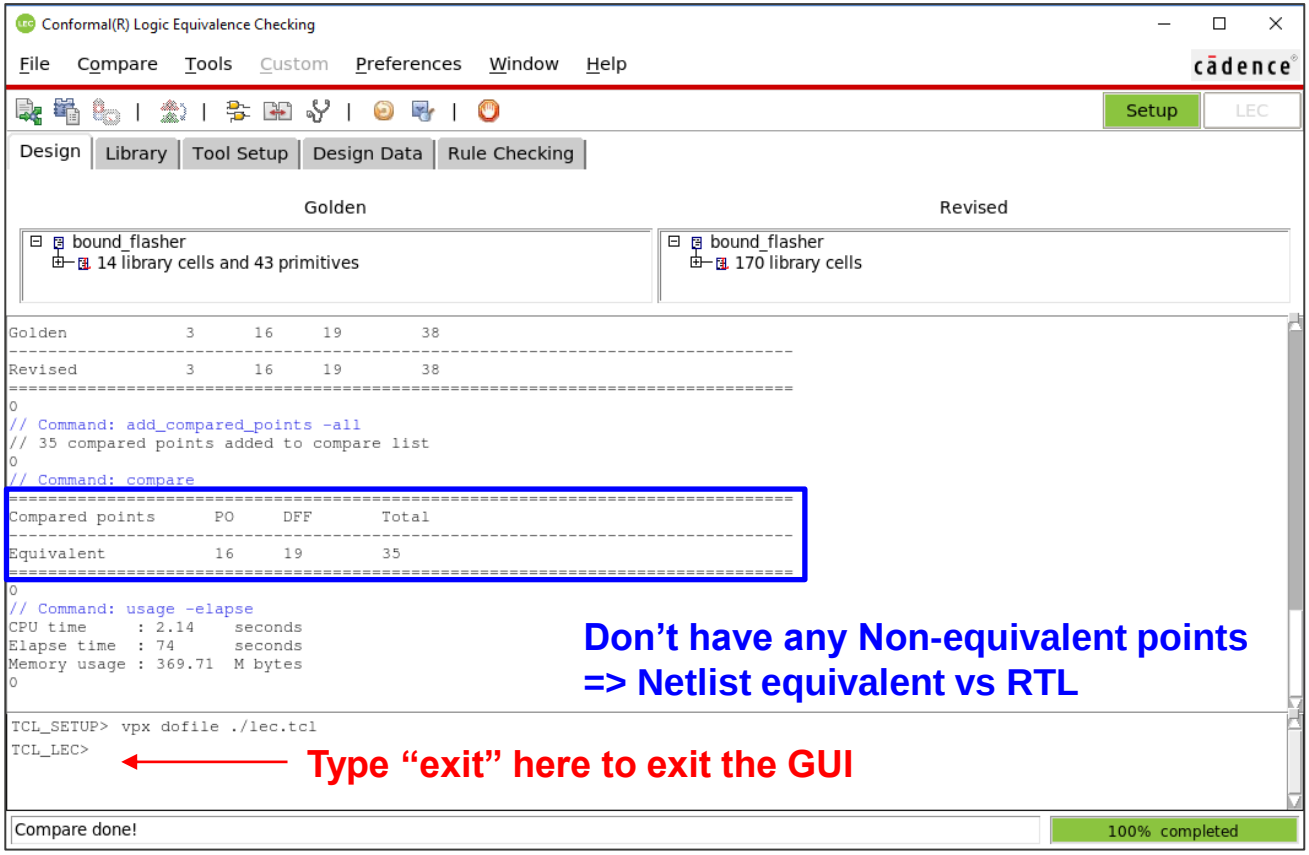
Setting license for using Conformal

Invoke Conformal

- **Step 6:** Execute `%> ./go_lec`

Perform LEC with Conformal (4/4)

- **Step 7:** It will automatically open the GUI and execute processes that we described in file *"lec.tcl"*



Debug Non-equivalent point (1/8)

- **Step 1:** Copy Netlist into this place:

```
%> rm -rf bound_flasher_m.v
%> cp -rf ../synthesis_env/Genus_BoundFlasher/LAB1/outputs??/bound_flasher_m.v ./
```

- **Step 2:** Confirm Netlist already copied (not link):

```
%> ll
```

```
[rvc@localhost lec_env]$ ll
total 24
-rw-rw-r--. 1 rvc rvc 12597 Jul 30 22:33 bound_flasher_m.v
lrwxrwxrwx. 1 rvc rvc    55 Jul 29 16:12 bound_flasher.v -> ../synthesis_env/Genus_BoundFlasher/RTL/bound_flasher.v
-rwxr-xr-x. 1 rvc rvc   121 Jul 30 11:27 go_lec
-rw-rw-r--. 1 rvc rvc  1004 Jul 30 11:32 lec.tcl
lrwxrwxrwx. 1 rvc rvc    48 Jul 29 16:13 slow.lib -> ../synthesis_env/Genus_BoundFlasher/LIB/slow.lib
```

Debug Non-equivalent point (2/8)

- **Step 3:** Modify Netlist to make a “bug” intentionally:

```
%> vi ./bound_flasher_m.v
```

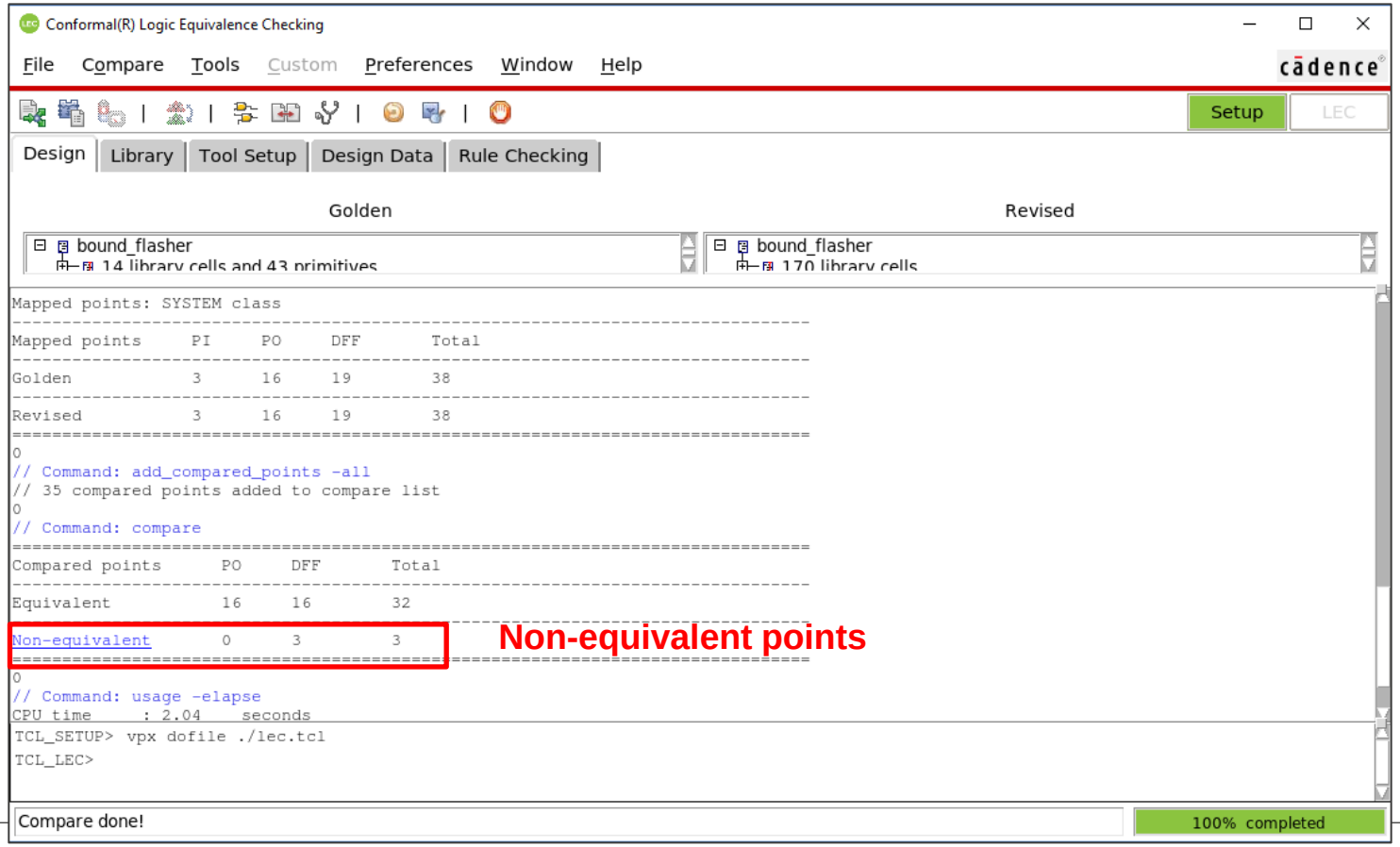
```
113   OA21X1 g3694__7482(.A0 (n_247), .A1 (n_39), .B0 (flick), .Y (n_41));
114   INVX3 g3717(.A (n_37), .Y (n_38));
115   NOR2X4 g3711__9945(.A (n_217), .B (n_25), .Y (n_24));
116   NAND2X1 g3723__2883(.A (n_7), .B (n_243), .Y (n_22));
117   NAND2X8 g3721__2346(.A (n_8), .B (flick), .Y (n_21));
118   NAND2X4 g3725__6417(.A (n_299), .B (n_18), .Y (n_37));
119   NAND2X6 g3715__2398(.A (n_247), .B (n_17), .Y (n_66));
120   INVX1 g3740(.A (n_17), .Y (n_16));
121   INVX1 g3744(.A (n_243), .Y (n_15));
122   //NHAN INVX2 g3742(.A (n_14), .Y (n_28));
123   BUFX2 g3742(.A (n_14), .Y (n_28));
124   INVX1 g3728(.A (n_25), .Y (n_13));
125   NAND2X2 g3713__5107(.A (state[0]), .B (n_9), .Y (n_53));
126   CLKAND2X8 g3716__6260(.A (n_29), .B (n_2), .Y (n_23));
127   CLKINVX6 g3729(.A (n_19), .Y (n_51));
128   NAND2X1 g3746__4319(.A (lamp[0]), .B (n_29), .Y (n_12));
129   NAND2X6 g3735__5526(.A (n_237), .B (flick), .Y (n_18));
130   INVX3 g3743(.A (n_9), .Y (n_14));
131   NOR2X6 g3738__6783(.A (n_238), .B (n_219), .Y (n_19));
132   CLKAND2X8 g3737__3680(.A (lamp[4]), .B (n_68), .Y (n_39));
```

Pick up 1 INV-cell randomly and change it to BUF-cell

- **Step 4:** Re-execute `%> ./go_lec`

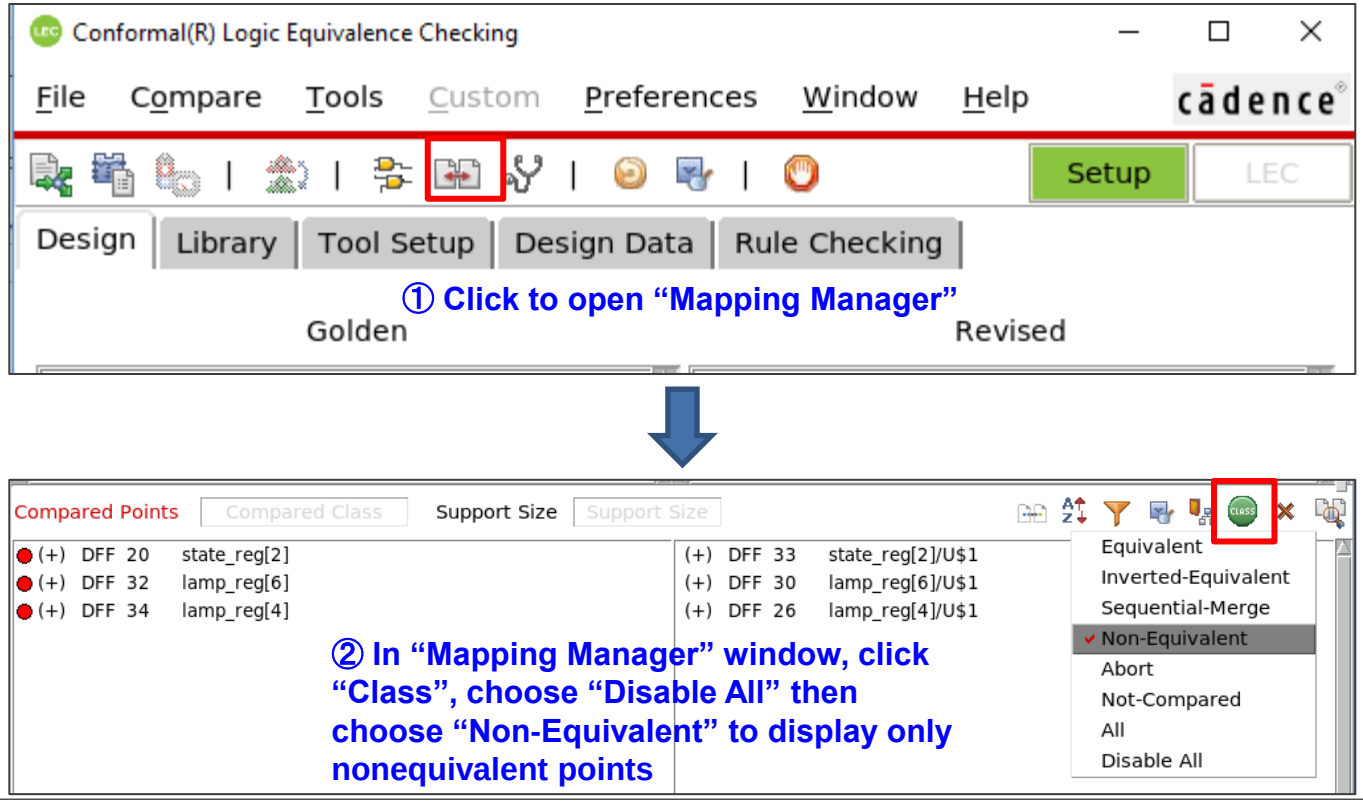
Debug Non-equivalent point (3/8)

- **Step 5:** When execution is done, “Non-equivalent” points will appear:

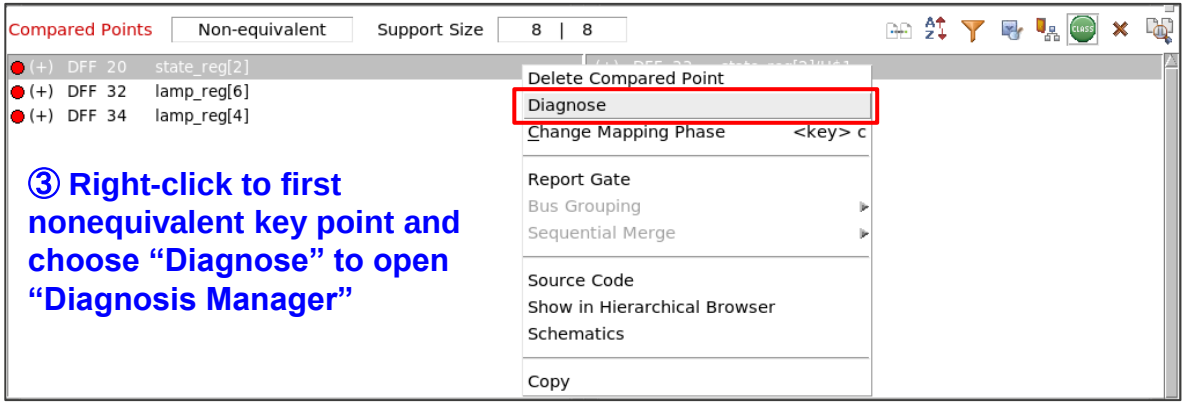


Debug Non-equivalent point (4/8)

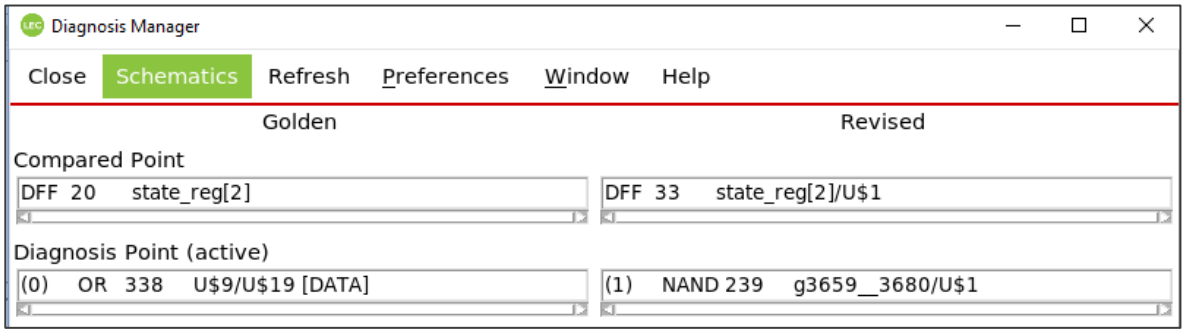
- **Step 6:** There are many ways for debugging nonequivalent points. The following part is the debugging example using "Diagnosis Manager" and "Schematics Viewer" in GUI:



Debug Non-equivalent point (5/8)

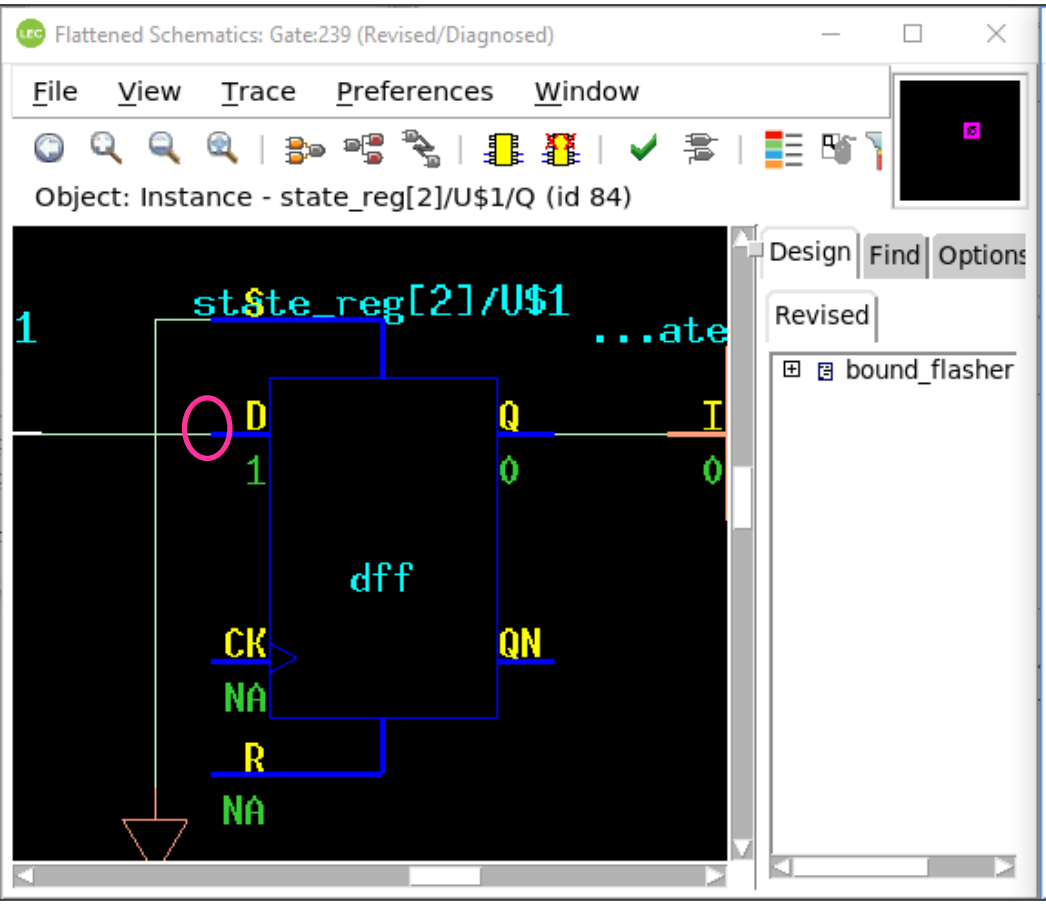


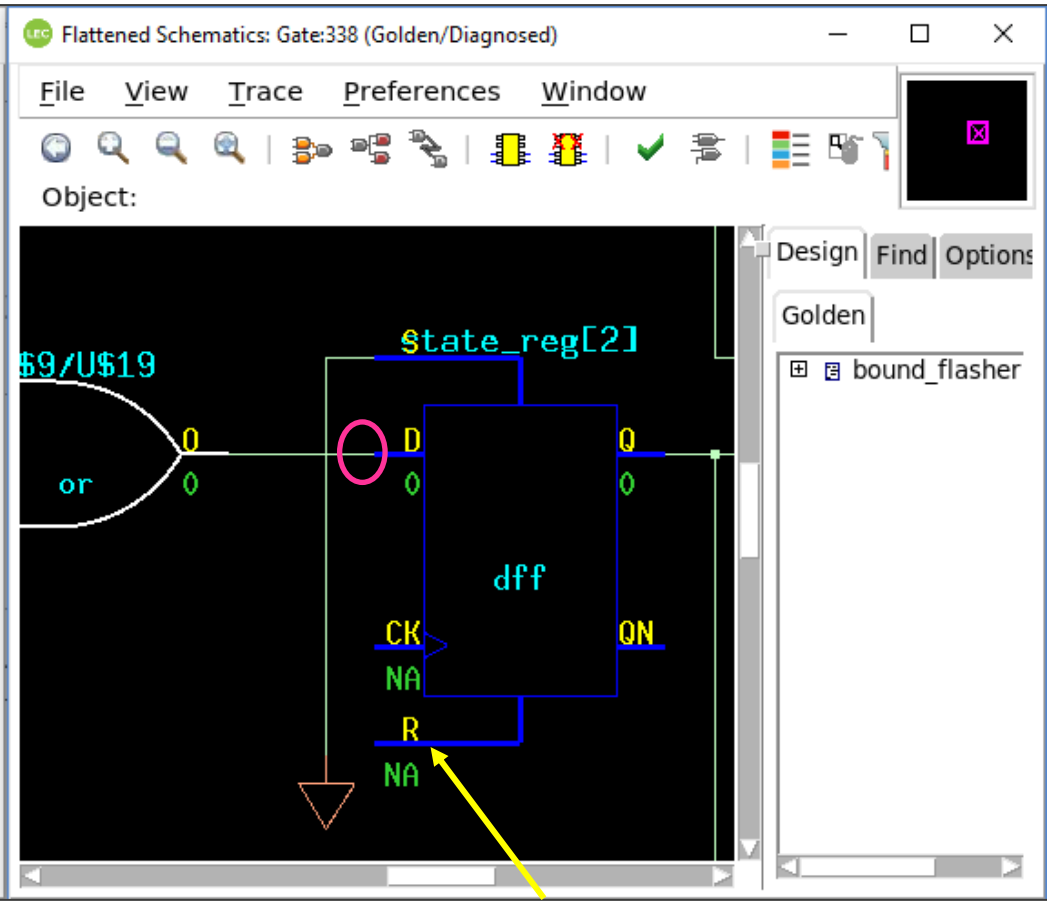
③ Right-click to first nonequivalent key point and choose “Diagnose” to open “Diagnosis Manager”



④ In “Diagnosis Manager” window, choose “Schematics” to open “Schematics Viewer”

Debug Non-equivalent point (6/8)

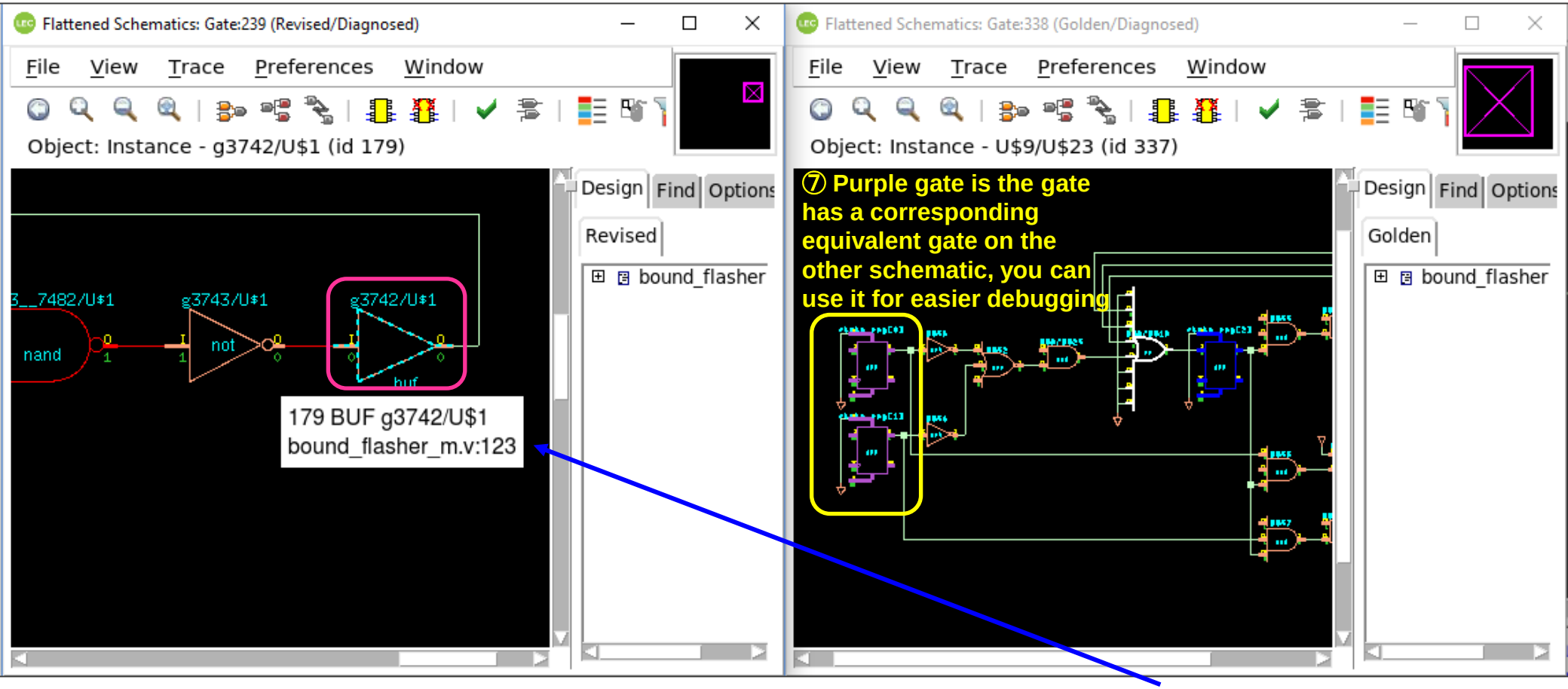




⑤ As you can see, on the Revised schematic (left), the end-point simulation (pink circle) is 1, while the Golden is 0

⑥ If you want to expand the logic cone at specific pin, right-click to this pin and choose “Fan-in Cone –> Open”

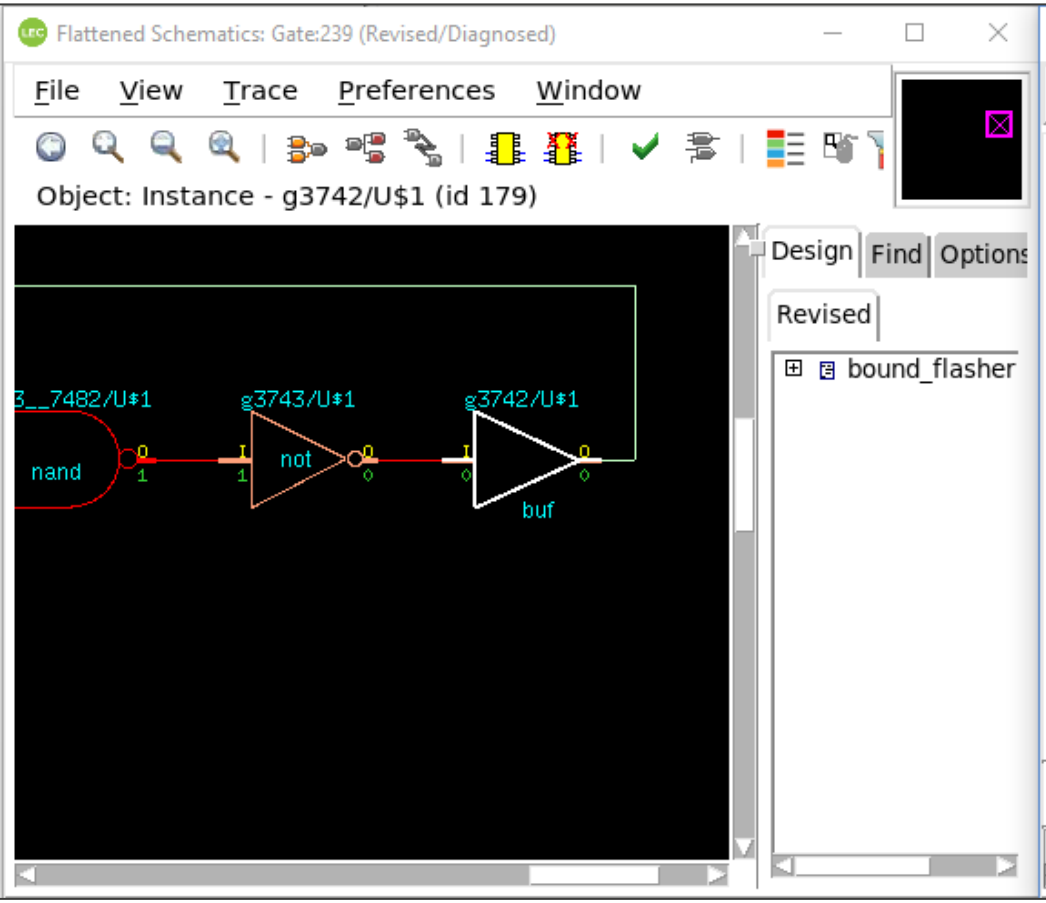
Debug Non-equivalent point (7/8)

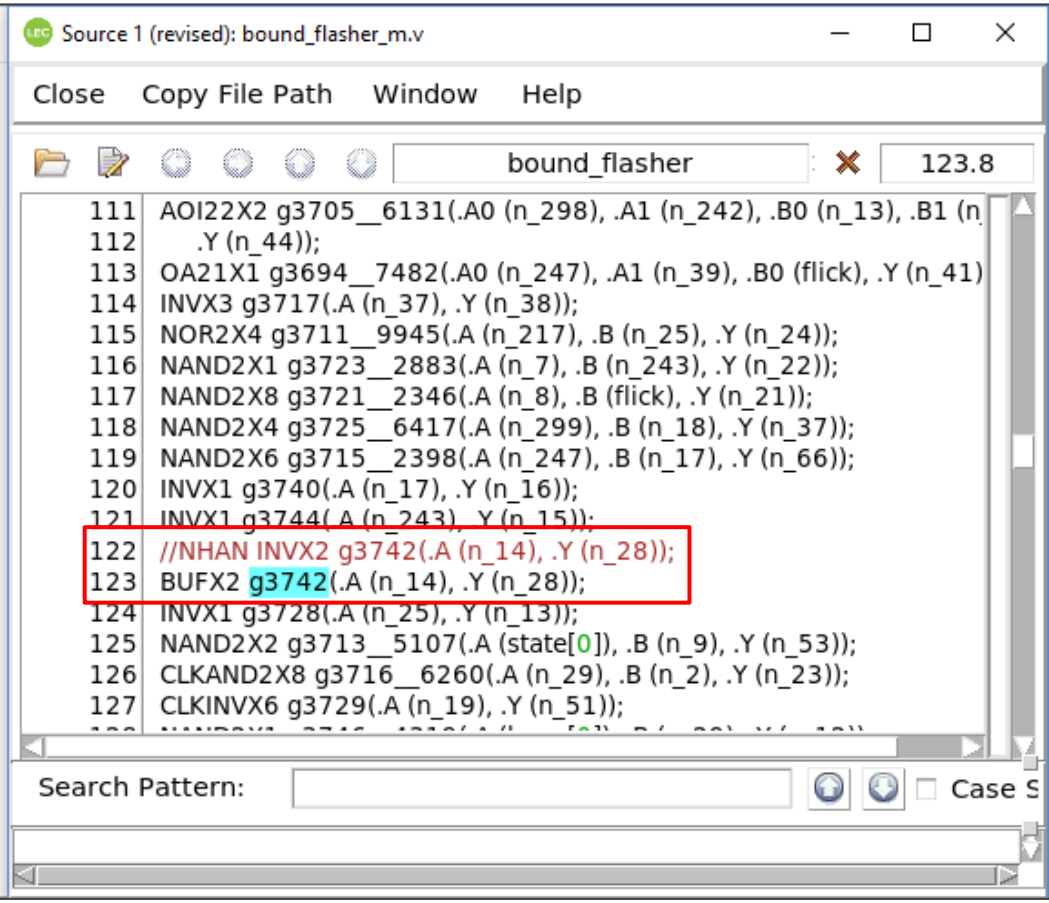


⑧ After trace back, you can find that the root cause is BUF-gate (pink rectangle) in Revised design

⑨ When you hover your mouse over any gate, it will show the corresponding line number in source code

Debug Non-equivalent point (8/8)





⑩ Double-click to the error gate (BUF gate) in Revised design, the corresponding source code will be opened

⑪ You can confirm and give the suitable solution such as re-synthesize, do ECO…

THANK YOU FOR YOUR LISTENING